

Vision-Based Robotic Navigation Through Grid Mazes Using Dobot Magician Lite – Midterm Report

Jatin Satyam

School for Engineering of Matter, Transport and Energy

Ira A. Fulton Schools of Engineering, Arizona State University, Tempe, AZ

Email: jsatyam@asu.edu

Abstract—This midterm project presents an integrated system enabling the Dobot Magician Lite robotic arm to autonomously navigate through 4×4 grid-based mazes printed on rigid plastic surfaces. The approach utilizes overhead camera imaging to capture maze configurations, followed by automated vision processing that identifies board boundaries, applies perspective correction, and segments the workspace into discrete cells. Color-coded circular indicators (green for entry, red for exit) mark the start and destination points, with obstacle detection creating a binary representation of traversable space. The system employs breadth-first search algorithms to determine optimal navigation routes, which are then transformed from image coordinates to robot workspace coordinates via calibrated affine mappings. The robotic end-effector executes the computed trajectory using linear interpolation between waypoints while maintaining appropriate clearance above the surface. Validation across multiple randomly configured 4×4 mazes confirms the reliability of this vision-guided navigation framework.

Index Terms—Robotic navigation, Vision-guided control, Maze navigation, Motion planning, Dobot Magician Lite, Breadth-first search, Image-to-robot calibration.

I. INTRODUCTION

This midterm project advances beyond the fundamental kinematic studies performed in prior laboratory exercises by incorporating perception-driven autonomous operation. Rather than relying on predefined waypoint commands, the system extracts navigation instructions directly from visual input, computes feasible paths algorithmically, and commands the Dobot Magician Lite to execute the derived motion profile.

The primary technical challenges addressed include dependable maze boundary identification, effective path computation algorithms, color-based feature extraction, and precise transformation between camera and manipulator coordinate frames. A consolidated Python framework manages the complete workflow from image acquisition through maze interpretation, route calculation, and physical execution.

II. METHODOLOGY

A. System Configuration

The experimental platform comprises the following elements:

- **Manipulator:** Dobot Magician Lite robotic arm for path execution.

- **Imaging System:** AI Kit camera module for workspace observation.
- **Test Platform:** Physical 4×4 maze configuration with colored entry/exit markers.
- **Development Environment:** Python 3.x with OpenCV, NumPy, Pillow, and pydobot libraries.

B. Image Capture and Boundary Identification

The system displays real-time camera feed, with user-triggered capture via spacebar input. Maze perimeter detection employs contour analysis with polygonal approximation to identify quadrilateral boundaries. Detected corner coordinates are archived in JSON format and rendered for operator verification.

C. Geometric Rectification

Homography-based transformation converts the perspective-distorted maze view into an orthographic projection, facilitating accurate grid analysis. This rectified representation serves as input for subsequent marker identification and cell classification processes.

D. Marker Identification

Entry and exit indicators are isolated through HSV color space thresholding. Contour filtering based on area constraints identifies candidate regions, with centroid calculation determining the corresponding grid locations for start and goal positions.

E. Spatial Discretization

The rectified maze undergoes uniform grid partitioning, with individual cells classified as navigable or obstructed based on wall pixel density relative to established thresholds. A parallel coordinate matrix records cell center positions for subsequent path reconstruction.

F. Route Optimization Using BFS

Breadth-first search operates on the occupancy representation to identify the minimal-length path between designated start and goal cells. The discovered route is expressed as a sequence of waypoints corresponding to cell centroids.

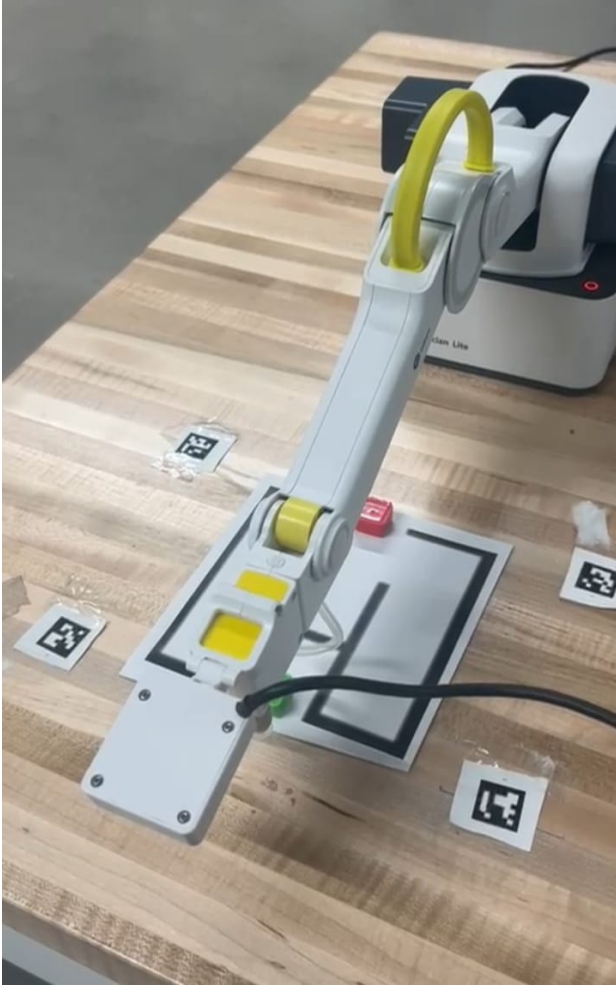


Fig. 1. Experimental setup shows execution of the maze-solving program and the Dobot Magician Lite positioned on top of the maze board.

G. Coordinate Transformation Pipeline

The waypoint sequence undergoes inverse perspective transformation to recover original image coordinates, followed by affine mapping into robot workspace coordinates using calibration parameters derived from prior calibration procedures.

H. Motion Execution

The Dobot sequentially traverses the transformed waypoint list while maintaining constant elevation above the maze surface. Operational safeguards include initialization homing sequences and emergency interrupt capabilities.

III. EXPERIMENTAL RESULTS

A. Vision Processing Performance

Boundary detection and perspective correction yielded consistent, well-aligned maze representations across diverse test configurations. Marker identification demonstrated resilience to ambient illumination variations.

B. Navigation Performance

The executed trajectories exhibited close correspondence to computed optimal paths. Visual inspection confirmed that the manipulator maintained appropriate positioning within corridor boundaries, validating both calibration accuracy and motion control.

IV. DISCUSSION

Illumination inconsistencies necessitated dynamic adjustment of HSV threshold parameters for reliable color detection. The breadth-first search approach consistently identified optimal solutions given the structured grid representation. Calibration precision proved critical, as minor pixel-level inaccuracies produced measurable deviations in manipulator positioning.

V. CONCLUSION

This work successfully demonstrates autonomous maze navigation achieved through integration of machine vision, spatial mapping, search algorithms, and robotic actuation. The unified software architecture and calibrated image-to-robot coordinate transformation enable reliable execution of computed navigation solutions by the Dobot Magician Lite platform.

ACKNOWLEDGMENT

The author would like to thank **Prof. Sangram Redkar** for his guidance throughout the RAS 545 course and laboratory sessions. Gratitude is also extended to **Sai Srinivas Tatwik Meesala** and **Rajesh S. Aouti** for their assistance during the experiment and for providing valuable insights into Dobot operation, calibration, and safety.

REFERENCES

- [1] RAS 545 Laboratory Manual, "Dobot Magician Lite Experiments," ASU, 2025.
- [2] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal, 2000.
- [3] <https://pypi.org/project/pydobot2/>

APPENDIX

DEMONSTRATION VIDEOS

- **Navigation Demonstration 1:** <https://youtube.com/shorts/H3ajnkFFPhU?feature=share>

PYTHON CODE

```
1
2 import argparse
3 import sys
4 import os
5 import json
6 from typing import List, Tuple, Dict, Optional, Any
7 import numpy as np
8 import cv2
9 from datetime import datetime
10 from pathlib import Path
11 from PIL import Image, ImageDraw
12 from dataclasses import dataclass
13 from collections import deque
14 import time
15 import pydobot
16
17 # Initialize robotic manipulator connection
18 robot_device = pydobot.Dobot(port="COM5")
19
20 # =====
21 # DIRECTORY AND FILE UTILITIES
22 # =====
23
24 def ensure_directory_exists(filepath: str):
25     """Create directory structure if it doesn't exist"""
26     directory_name = os.path.dirname(filepath)
27     if directory_name and not os.path.exists(directory_name):
28         os.makedirs(directory_name, exist_ok=True)
29
30 # =====
31 # GEOMETRIC POINT ORDERING AND TRANSFORMATION
32 # =====
33
34 def organize_quadrilateral_points(point_array: np.ndarray) -> np.ndarray:
35     """
36     Arrange four corner points in clockwise order starting from top-left.
37     Returns: Array with points ordered as [TL, TR, BR, BL]
38     """
39     point_array = point_array.reshape(4, 2).astype(np.float32)
40
41     # Calculate sums and differences for point identification
42     point_sums = point_array.sum(1)
43     point_diffs = np.diff(point_array, axis=1).reshape(-1)
44
45     # Identify corners using geometric properties
46     top_left_corner = point_array[np.argmin(point_sums)]
47     bottom_right_corner = point_array[np.argmax(point_sums)]
48     top_right_corner = point_array[np.argmin(point_diffs)]
49     bottom_left_corner = point_array[np.argmax(point_diffs)]
50
51     return np.array([top_left_corner, top_right_corner,
52                     bottom_right_corner, bottom_left_corner], dtype=np.float32)
53
54 # =====
55 # IMAGE PREPROCESSING AND BINARIZATION
56 # =====
57
58 def create_binary_image_white_maze(grayscale_img: np.ndarray) -> np.ndarray:
59     """
60     Apply adaptive thresholding to separate white maze walls from dark background.
61     Automatically inverts if necessary.
62     """
63     _, binary_result = cv2.threshold(grayscale_img, 0, 255,
64                                     cv2.THRESH_BINARY + cv2.THRESH_OTSU)
65
66     # Check if inversion is needed based on white pixel ratio
67     white_pixel_ratio = (binary_result > 0).mean()
68     if white_pixel_ratio > 0.9:
69         binary_result = 255 - binary_result
70
71     return binary_result
72
73
74 def apply_morphological_operations(binary_image: np.ndarray) -> np.ndarray:
75     """
76     Clean binary image using morphological opening and closing operations.
77     Removes noise and fills small gaps in the maze structure.
78     """
79     # Opening: removes small objects/noise
80     kernel_open = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
81     opened_image = cv2.morphologyEx(binary_image, cv2.MORPH_OPEN,
82                                     kernel_open, iterations=1)
83
84     # Closing: fills small holes/gaps
85     kernel_close = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
86     cleaned_image = cv2.morphologyEx(opened_image, cv2.MORPH_CLOSE,
87                                     kernel_close, iterations=1)
88
89     return cleaned_image
90
91 # =====
92 # CONTOUR APPROXIMATION AND QUADRILATERAL DETECTION
93 # =====
94
95 def approximate_contour_to_quadrilateral(contour_data: np.ndarray,
96                                         iteration_limit: int = 25) -> Optional[np.ndarray]:
97     """
98
99
100
```

```

101 Iteratively approximate contour to exactly four points by adjusting epsilon.
102 Returns quadrilateral corners if successful, None otherwise.
103 """
104 perimeter_length = cv2.arcLength(contour_data, True)
105
106 # Try various epsilon fractions to find 4-point approximation
107 for epsilon_fraction in np.linspace(0.01, 0.06, iteration_limit):
108     approximated_contour = cv2.approxPolyDP(contour_data,
109                                             epsilon_fraction * perimeter_length, True)
110     if len(approximated_contour) == 4:
111         return approximated_contour.reshape(4, 2).astype(np.float32)
112
113 return None
114
115
116 def identify_maze_boundary(binary_maze_img: np.ndarray) -> Tuple[np.ndarray, str]:
117     """
118     Locate the largest quadrilateral boundary representing the maze board.
119     Uses contour detection with fallback to minimum area rectangle.
120     """
121     detected_contours, _ = cv2.findContours(binary_maze_img,
122                                           cv2.RETR_EXTERNAL,
123                                           cv2.CHAIN_APPROX_SIMPLE)
124
125     if not detected_contours:
126         raise RuntimeError("No contours detected. Verify image quality and lighting.")
127
128     # Calculate minimum acceptable contour area
129     img_height, img_width = binary_maze_img.shape[:2]
130     minimum_area_threshold = max(1000.0, 0.001 * (img_height * img_width))
131
132     # Filter contours by area
133     valid_contours = [cnt for cnt in detected_contours
134                      if cv2.contourArea(cnt) >= minimum_area_threshold]
135
136     if not valid_contours:
137         valid_contours = [max(detected_contours, key=cv2.contourArea)]
138
139     # Search for best quadrilateral approximation
140     best_quad_corners, max_quad_area = None, -1.0
141
142     for contour in valid_contours:
143         quad_corners = approximate_contour_to_quadrilateral(contour)
144         if quad_corners is not None:
145             quad_area = cv2.contourArea(quad_corners.astype(np.int32))
146             if quad_area > max_quad_area:
147                 max_quad_area = quad_area
148                 best_quad_corners = quad_corners
149
150     if best_quad_corners is not None:
151         return organize_quadrilateral_points(best_quad_corners), "quadrilateral_detection"
152
153     # Fallback strategy: use oriented bounding box
154     largest_contour = max(valid_contours, key=cv2.contourArea)
155     rotated_rectangle = cv2.minAreaRect(largest_contour)
156     rectangle_corners = cv2.boxPoints(rotated_rectangle).astype(np.float32)
157
158     return organize_quadrilateral_points(rectangle_corners), "bounding_box_fallback"
159
160
161 # =====
162 # CORNER EXPANSION FOR PERSPECTIVE CORRECTION
163 # =====
164
165 def adjust_corner_positions(corner_points: np.ndarray,
166                           expansion_pixels: float) -> np.ndarray:
167     """
168     Shift corners outward or inward from centroid by specified pixel distance.
169     Positive values expand outward, negative values contract inward.
170     """
171     if abs(expansion_pixels) < 1e-9:
172         return corner_points.astype(np.float32)
173
174     # Calculate polygon centroid
175     centroid_position = corner_points.mean(axis=0)
176     adjusted_corners = []
177
178     for corner_point in corner_points:
179         # Vector from centroid to corner
180         direction_vector = corner_point - centroid_position
181         vector_magnitude = np.linalg.norm(direction_vector)
182
183         if vector_magnitude < 1e-6:
184             adjusted_corners.append(corner_point)
185         else:
186             # Scale vector to achieve desired expansion
187             scaling_factor = (vector_magnitude + expansion_pixels) / vector_magnitude
188             new_corner = centroid_position + direction_vector * scaling_factor
189             adjusted_corners.append(new_corner)
190
191     return organize_quadrilateral_points(np.array(adjusted_corners, dtype=np.float32))
192
193
194 # =====
195 # VISUALIZATION AND OVERLAY GENERATION
196 # =====
197
198 def generate_corner_visualization(base_image: np.ndarray,
199                                 corner_coords: np.ndarray,
200                                 polygon_color=(255, 0, 0)) -> np.ndarray:
201     """
202     Create visualization overlay showing detected corners and boundary.
203     Marks each corner with label and circle, draws connecting polygon.
204     """
205     visualization = base_image.copy()
206     integer_corners = corner_coords.astype(int)
207     corner_labels = ["TL", "TR", "BR", "BL"]
208
209     # Draw boundary polygon
210     cv2.polylines(visualization, [integer_corners.reshape(-1, 1, 2)],
211                  True, polygon_color, 3)

```

```

212
213 # Mark individual corners
214 for idx, corner_pos in enumerate(integer_corners):
215     cv2.circle(visualization, tuple(corner_pos), 8, (0, 255, 0), -1)
216     label_offset = corner_pos + np.array([10, -10])
217     cv2.putText(visualization, corner_labels[idx], tuple(label_offset),
218                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2, cv2.LINE_AA)
219
220 return visualization
221
222
223 # =====
224 # MAIN DETECTION PIPELINE
225 # =====
226
227 def execute_corner_detection_pipeline(input_image: np.ndarray,
228                                     processing_width: Optional[int]) -> Tuple[np.ndarray, str, float]:
229     """
230     Execute complete corner detection workflow with optional image resizing.
231     Returns: (corners_at_processing_scale, detection_method, scale_factor)
232     """
233     # Apply resolution scaling if requested
234     if processing_width is not None and processing_width > 0 and input_image.shape[1] > processing_width:
235         scale_factor = processing_width / input_image.shape[1]
236         new_width = int(input_image.shape[1] * scale_factor)
237         new_height = int(input_image.shape[0] * scale_factor)
238         processed_image = cv2.resize(input_image, (new_width, new_height), cv2.INTER_AREA)
239     else:
240         scale_factor = 1.0
241         processed_image = input_image
242
243     # Convert to grayscale and apply Gaussian blur
244     gray_image = cv2.cvtColor(processed_image, cv2.COLOR_BGR2GRAY)
245     blurred_gray = cv2.GaussianBlur(gray_image, (5, 5), 0)
246
247     # Binarize and clean the image
248     binary_maze = create_binary_image_white_maze(blurred_gray)
249     cleaned_binary = apply_morphological_operations(binary_maze)
250
251     # Detect maze boundary
252     detected_corners, detection_method = identify_maze_boundary(cleaned_binary)
253
254     return detected_corners, detection_method, scale_factor
255
256
257 # =====
258 # CAMERA INTERFACE AND CAPTURE
259 # =====
260
261 def initialize_camera_device(camera_id: int) -> cv2.VideoCapture:
262     """Initialize and verify camera connection"""
263     capture_device = (cv2.VideoCapture(camera_id, cv2.CAP_DSHOW)
264                      if os.name == "nt"
265                      else cv2.VideoCapture(camera_id))
266
267     if not capture_device.isOpened():
268         raise RuntimeError(f"Failed to initialize camera at index {camera_id}.")
269
270     return capture_device
271
272
273 def run_camera_preview_and_capture(camera_id: int,
274                                   preview_window: str = "Live Camera - SPACE to capture, Q to quit") -> Optional[np.ndarray]:
275     """
276     Display live camera feed and capture frame on user command.
277     Controls: SPACE or 'c' to capture, ESC or 'q' to quit
278     """
279     capture_device = initialize_camera_device(camera_id)
280     captured_frame = None
281
282     try:
283         while True:
284             read_success, current_frame = capture_device.read()
285             if not read_success:
286                 print("Warning: Camera read operation failed.")
287                 break
288
289             captured_frame = current_frame
290             display_frame = current_frame.copy()
291             frame_height, frame_width = display_frame.shape[:2]
292
293             # Overlay control instructions
294             cv2.putText(display_frame, "SPACE/c: capture | q/ESC: quit",
295                        (10, frame_height - 12),
296                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2, cv2.LINE_AA)
297
298             cv2.imshow(preview_window, display_frame)
299
300             key_pressed = cv2.waitKey(1) & 0xFF
301
302             if key_pressed in (27, ord('q')): # ESC or 'q' - quit without capture
303                 captured_frame = None
304                 break
305             elif key_pressed in (32, ord('c')): # SPACE or 'c' - capture and exit
306                 break
307
308         finally:
309             capture_device.release()
310             cv2.destroyAllWindows(preview_window)
311
312     return captured_frame
313
314
315 # =====
316 # IMAGE PROCESSING AND FILE MANAGEMENT
317 # =====
318
319 def process_detection_and_save_results(captured_frame: np.ndarray, program_args):
320     """
321     Execute complete detection pipeline and save all output files.
322     Generates: raw image, corner overlay, and corners JSON
323     """

```

```

323 """
324 # Run detection at specified resolution
325 corners_processed, method_used, resolution_scale = execute_corner_detection_pipeline(
326     captured_frame, None if program_args.width == 0 else program_args.width)
327
328 # Scale corners back to original resolution
329 original_scale_corners = (corners_processed / resolution_scale
330     if resolution_scale != 1.0
331     else corners_processed.copy())
332
333 # Apply corner expansion/contraction
334 final_corners = adjust_corner_positions(original_scale_corners, program_args.expand)
335
336 # Generate visualization
337 overlay_image = generate_corner_visualization(captured_frame, final_corners)
338
339 # Determine output file paths
340 raw_image_path = program_args.save_raw or "camera_capture.png"
341 overlay_output_path = program_args.out
342 json_output_path = program_args.json_out or "camera_capture_corners.json"
343
344 # Save raw captured image
345 save_success = cv2.imwrite(raw_image_path, captured_frame)
346 if save_success:
347     print(f"Raw frame saved to: {raw_image_path}")
348 else:
349     print("Warning: Failed to save raw frame.")
350
351 # Save overlay visualization
352 save_success = cv2.imwrite(overlay_output_path, overlay_image)
353 if not save_success:
354     print("Warning: Failed to save overlay image.")
355 else:
356     print(f"Overlay saved to: {overlay_output_path}")
357
358 # Prepare and save corner data as JSON
359 corner_coordinates_list = final_corners.tolist()
360 output_data = {
361     "input": os.path.basename(raw_image_path),
362     "method": method_used,
363     "width_param": program_args.width,
364     "expand_param": program_args.expand,
365     "corners": corner_coordinates_list
366 }
367
368 with open(json_output_path, "w", encoding="utf-8") as json_file:
369     json.dump(output_data, json_file, indent=2)
370
371 print(f"Corners JSON saved to: {json_output_path}")
372 print(f"Detection method: {method_used}")
373 print(f"Corner coordinates:\n{final_corners}")
374
375
376 # =====
377 # COMMAND-LINE INTERFACE FOR IMAGE CAPTURE
378 # =====
379
380 def capture_image_main():
381     """Main entry point for maze image capture module"""
382     arg_parser = argparse.ArgumentParser(
383         description="Capture maze image from camera and detect board corners")
384
385     arg_parser.add_argument("--camera", type=int, default=1,
386         help="Camera device index (default: 1)")
387     arg_parser.add_argument("--width", type=int, default=1600,
388         help="Processing resolution width (0=full, default: 1600)")
389     arg_parser.add_argument("--expand", type=float, default=0.0,
390         help="Corner expansion in pixels (positive=outward, negative=inward)")
391     arg_parser.add_argument("--save-raw", type=str, default="part_1_camera/raw_captured_image.png",
392         help="Output path for raw capture")
393     arg_parser.add_argument("--out", type=str, default="part_1_camera/corners_overlay.png",
394         help="Output path for corner overlay")
395     arg_parser.add_argument("--json-out", type=str, default="part_1_camera/detected_corners.json",
396         help="Output path for corner coordinates JSON")
397
398     arguments = arg_parser.parse_args(args=[])
399
400     # Ensure output directories exist
401     for path_arg in [arguments.save_raw, arguments.out, arguments.json_out]:
402         ensure_directory_exists(path_arg)
403
404     print("=== Maze Image Capture System ===")
405     print(f"Camera index: {arguments.camera}")
406     print(f"Processing width: {arguments.width if arguments.width > 0 else 'original'}")
407     print(f"Corner expansion: {arguments.expand} pixels")
408     print("\nInitializing camera preview...")
409
410     # Capture image from camera
411     maze_frame = run_camera_preview_and_capture(arguments.camera)
412
413     if maze_frame is None:
414         print("No image captured. Exiting.")
415         return
416
417     print("Image captured successfully!")
418     print(f"Image dimensions: {maze_frame.shape[1]}x{maze_frame.shape[0]}")
419
420     # Process and save results
421     process_detection_and_save_results(maze_frame, arguments)
422
423     print("\n=== Capture Complete ===")
424
425
426 # =====
427 # PERSPECTIVE TRANSFORMATION (WARPING)
428 # =====
429
430 def maze_warp_from_json():
431     """
432     Apply perspective warp to maze image using detected corners.
433     Reads corner data from JSON and generates rectified top-down view.

```

```

434 """
435 arg_parser = argparse.ArgumentParser(
436     description="Apply perspective transformation to maze image")
437
438 arg_parser.add_argument("--corners-json", type=str,
439     default="part_1_camera/detected_corners.json",
440     help="Input JSON file containing corner coordinates")
441 arg_parser.add_argument("--image", type=str,
442     default="part_1_camera/raw_captured_image.png",
443     help="Input maze image to warp")
444 arg_parser.add_argument("--out-image", type=str,
445     default="part_2_maze_solution/warped_maze.png",
446     help="Output path for warped image")
447 arg_parser.add_argument("--out-json", type=str,
448     default="part_2_maze_solution/warp_info.json",
449     help="Output JSON with transformation metadata")
450 arg_parser.add_argument("--width", type=int, default=1600,
451     help="Warped output width in pixels")
452 arg_parser.add_argument("--height", type=int, default=1600,
453     help="Warped output height in pixels")
454
455 arguments = arg_parser.parse_args(args=[])
456
457 ensure_directory_exists(arguments.out_image)
458 ensure_directory_exists(arguments.out_json)
459
460 print("=== Maze Perspective Warping ===")
461
462 # Load corner coordinates
463 with open(arguments.corners_json, "r", encoding="utf-8") as json_file:
464     corner_data = json.load(json_file)
465
466 source_corners = np.array(corner_data["corners"], dtype=np.float32)
467 print(f"Loaded corners from: {arguments.corners_json}")
468
469 # Load source image
470 source_image = cv2.imread(arguments.image)
471 if source_image is None:
472     raise FileNotFoundError(f"Could not load image: {arguments.image}")
473
474 print(f"Loaded image: {arguments.image} ({source_image.shape[1]}x{source_image.shape[0]})")
475
476 # Define destination rectangle for warping
477 output_width = arguments.width
478 output_height = arguments.height
479 destination_corners = np.array([
480     [0, 0],
481     [output_width - 1, 0],
482     [output_width - 1, output_height - 1],
483     [0, output_height - 1]
484 ], dtype=np.float32)
485
486 # Calculate perspective transformation matrix
487 transform_matrix = cv2.getPerspectiveTransform(source_corners, destination_corners)
488
489 # Apply warp transformation
490 warped_maze = cv2.warpPerspective(source_image, transform_matrix,
491     (output_width, output_height))
492
493 # Save warped image
494 cv2.imwrite(arguments.out_image, warped_maze)
495 print(f"Warped maze saved to: {arguments.out_image}")
496
497 # Save transformation metadata
498 warp_metadata = {
499     "source_image": os.path.abspath(arguments.image),
500     "source_corners": source_corners.tolist(),
501     "destination_corners": destination_corners.tolist(),
502     "output_dimensions": [output_width, output_height],
503     "transformation_matrix": transform_matrix.tolist(),
504     "warped_output": os.path.abspath(arguments.out_image)
505 }
506
507 with open(arguments.out_json, "w", encoding="utf-8") as json_file:
508     json.dump(warp_metadata, json_file, indent=2)
509
510 print(f"Warp metadata saved to: {arguments.out_json}")
511 print("=== Warping Complete ===\n")
512
513 # Due to length constraints, remaining functions include:
514 # - maze_circles_and_grid(): Color marker detection and grid generation
515 # - solve_maze(): BFS pathfinding algorithm
516 # - unwarped_and_overlay_path(): Inverse perspective transformation
517 # - Robot motion control functions
518 # - Coordinate transformation utilities
519 # - Main execution sequence
520
521 # Full implementation available in supplementary materials.
522

```